

Service Overview

ภาพรวมของบริการ (Service Overview)

EmergencyResourceRequest Service

1. Service Owner

นายชนกันต์ ทองรอง รหัสนักศึกษา 6609611816 ภาคปกติ

2. Service Purpose

EmergencyResourceRequest Service เป็นบริการที่รับผิดชอบการรับเรื่อง รวบรวม และจัดการคำร้องขอทรัพยากรความช่วยเหลือ เช่น อาหาร น้ำ ยา จากผู้ประสบภัยและหน่วยงานส่วนหน้า โดยทำหน้าที่แปลงข้อมูลคำร้องขอให้อยู่ในรูปแบบที่มีโครงสร้าง (Structured Data) เพื่อส่งต่อให้ทีมกู้ภัยนำไปใช้วางแผนการกระจายสิ่งของได้อย่างมีประสิทธิภาพ

3. Pain Point ที่แก้ไข

ในสถานการณ์ภัยพิบัติ ข้อมูลการขอความช่วยเหลือมักกระจายไม่ระบุพิกัดที่ชัดเจน และมีการแจ้งซ้ำซ้อนเนื่องจากอินเทอร์เน็ตไม่เสถียร ทำให้หน่วยกู้ภัยไม่ทราบว่าพื้นที่ใดมีความต้องการเร่งด่วน บริการนี้จึงเข้ามาช่วยจัดระเบียบข้อมูล แยกแยะระดับความสำคัญ และป้องกันการบันทึกข้อมูลซ้ำซ้อน เพื่อให้ความช่วยเหลือไปถึงผู้ที่ต้องการจริงๆ ได้อย่างรวดเร็ว

3. Target Users

1. Citizen (ประชาชนผู้ประสบภัย): ผ่านทางแอปพลิเคชันหรือเว็บ Frontend
2. Rescue Team: เจ้าหน้าที่ลงพื้นที่หรือทีมกู้ภัยที่ต้องการดูรายการคำขอและกดรับงาน

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ):
 - รับและจัดเก็บข้อมูลคำร้องขอความช่วยเหลือและทรัพยากร (Resource Requests)
 - จัดการสถานะการดำเนินการของคำร้องขอ
 - ตรวจสอบและคัดกรองข้อมูลคำร้องขอซ้ำซ้อน (Duplicate Detection / Idempotency)
 - จัดเก็บพิกัด (Location) และระดับความเร่งด่วน (Priority) ของแต่ละคำขอ
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ):
 - การจัดการคลังสินค้าหรือสต็อกสิ่งของบริจาค (Inventory Management)
 - การจัดการข้อมูลทีมกู้ภัยหรือการนำทางรถบรรทุก (Fleet Routing)
 - การจัดการข้อมูลภาพรวมของเหตุการณ์ภัยพิบัติ (Incident Master Data)

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การตรวจจบข้อมูลซ้ำซ้อน (Idempotency): หากมีคำร้องขอที่มีข้อมูลผู้แจ้งและพิกัดเดิมส่งเข้ามาในเวลาใกล้เคียงกัน บริการจะปฏิเสธหรือรวมเป็นคำขอเดียวโดยอัตโนมัติ
- การอัปเดตสถานะ: เมื่อมีการระบุตัวทีมกู้ภัย (`delivery_team_id`) เข้ามาในระบบ บริการสามารถปรับสถานะจาก **NEW** เป็น **IN_PROGRESS** ได้ทันที

การตัดสินใจอิงจาก:

- ความถูกต้องของพิกัด (Location validation)
- ความสมบูรณ์ของรายการสิ่งของที่ขอ (Items payload)

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด โดยไม่ต้องรอ/ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

6. Owned Data

- **Resource Request Master Data (ข้อมูลคำร้องขอ):** ข้อมูลรายละเอียดคำขอ เช่น **items, requester** (ผู้แจ้ง), **location, priority** เป็นข้อมูลแกนหลักที่เกิดจาก Action ภายในบริการนี้ จึงต้องเป็นเจ้าของเพื่อให้แก้ไขและดึงข้อมูลได้อย่างรวดเร็ว
- **Request Operational State (สถานะการดำเนินการ):** ข้อมูล **status (NEW, IN_PROGRESS, CLOSED)** และ **time_requested** เป็น state ที่เชื่อมโยงกับ Lifecycle ของคำร้องขอ บริการนี้จึงต้องรับผิดชอบเพื่อป้องกันไม่ให้ Service อื่นมาแก้ไขสถานะโดยพลการ
- **Idempotency Keys / Request Logs (ข้อมูลป้องกันการซ้ำซ้อน):** ข้อมูลสำหรับตรวจสอบประวัติการส่ง Request ซ้ำซ้อน ต้องอยู่ภายใต้บริการนี้เพื่อรับประกันความถูกต้องของการสร้างข้อมูลใหม่

7. Linked Data (Reference Only)

- **incident_id:** อ้างอิงเหตุการณ์ภัยพิบัติจาก Incident Service เพื่อระบุว่าคำขอนี้เกิดในภัยพิบัติรอบไหน
- **delivery_team_id:** อ้างอิงทีมกู้ภัยจาก Rescue Team Service (ระบบจะไม่เก็บชื่อหรือเบอร์โทรทีมกู้ภัยไว้ที่นี่ แต่จะเก็บแค่ ID)
- **item_id:** อ้างอิงถึงสินค้าที่มีอยู่ใน stock (จะอยู่ใน field jsonb items) โดยจะอิงไปถึง service ที่เกี่ยวข้องกับจัดการ stock

8. Non-Functional Requirements

- **Idempotency:** API สำหรับสร้างคำขอ (**POST**) ต้องรองรับ Idempotency Key ป้องกันการสร้างข้อมูลซ้ำเมื่อผู้ประสบกดส่งซ้ำเพราะเน็ตหลุด
- **Availability:** เนื่องจากเป็นระบบที่ใช้งานในภาวะฉุกเฉิน API ฝั่งรับเรื่อง (Create Request) ต้องสามารถทำงานได้ (High Availability) แม้ Service อื่นที่เกี่ยวข้องจะล่ม
- **Data Integrity & Concurrency:** หากมีทีมกู้ภัย 2 ทีมพยายามกดรับเคสเดียวกันพร้อมกัน (อัปเดตสถานะเป็น **IN_PROGRESS**) ระบบต้องมีกลไก Optimistic Locking ป้องกันไม่ให้รับเคสซ้ำซ้อน
- **Security:** API Endpoint สำหรับดึงข้อมูลไปแสดงผลหรืออัปเดตสถานะ (ที่ใช้โดยทีมกู้ภัย) ต้องมีการยืนยันตัวตน (Authentication) เพื่อป้องกันข้อมูลส่วนบุคคลของผู้ประสบภัยรั่วไหล

Sync Contract

Synchronous Function Contract

Base URL: *http://TBD.com*

API Contract #1: Create Resource Request

ข้อมูลทั่วไป

- **Name:** Create a new resource request
- **Method:** POST
- **Path:** **/v1/request**
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับสร้างคำร้องขอทรัพยากรและความช่วยเหลือใหม่จากผู้ประสภักย์หรือเจ้าหน้าที่ส่วนหน้า

Request

Query Params

-

Headers

- **Content-Type:** application/json
- **Idempotency-Key:** <uuid>

Body:

```
{
  "items": [
    {
      "id": "1",
      "amount": 1
    }
  ],
  "extra_items": [
    {
      "name": "ยาหม่อง", "amount": 3,
    }
  ],
  "from": {
    "name": "Somchai Maipong",
    "location": {
      "address": "32/1 Phichit",

```

```
{
  "description": "Behind 7-11",
  "latitude": 16.4425,
  "longitude": 100.3488
},
{
  "contact": {
    "phone": "012345678"
  }
},
}
```

Response

Success: 201

```
{ "id": "req-1234-uuid", "status": "NEW", "requested_at": "2026-01-30T14:25:00Z" }
```

Error: 400

```
{
  "code": "VALIDATION_ERROR",
  "message": "from.location.latitude and longitude are required"
}
```

Dependency / Reliability

- ไม่เรียก service อื่นแบบ Synchronous
- ต้องทำ Idempotency check จาก Header
- Timeout: 10s

API Contract #2: List Requests

ข้อมูลทั่วไป

- **Name:** List requests by incident and status
- **Method:** GET
- **Path:** **/v1/requests**
- **Type:** Synchronous

คำอธิบาย

ใช้ดึงข้อมูลรายการคำร้องขอ เพื่อให้ระบบส่วนกลาง (Dispatcher) หรือทีมกู้ภัย ใช้ดูว่ามีเคสไหนบ้างที่ยังไม่มีคนไปช่วยเหลือ หรือเพื่อจัดพิกัดการส่งของ

Request

Query Params

- **incident_id** (uuid, required)
- **status** (string, optional)
- **priority** (string, optional)

Headers

- Content-Type: application/json
- Authorization: Bearer <token>

Body

Validation

- capacityAvailable $\geq$ 0
- status $\in$ {OPEN, FULL, CLOSED}
- incidentId required

Response

Success: 200

```
[
  {
    "id": "uuid",
    "items": [
      {
        "id": "1",
        "amount": 1
      }
    ],
    "extra_items": [
      {
        "name": "ยาพาราเซตามอล", "amount": 3,
      }
    ],
    "from": {
      "name": "Somchai Maipong",
      "location": {
        "address": "32/1 Phichit",
        "description": "Behind 7-11",
        "latitude": 16.4425,
        "longitude": 100.3488
      },
      "contact": {
        "phone": "012345678"
      }
    }
  }
]
```

```
    }
  },
}
]
```

Error: 404

```
{
  "code": "VALIDATION_ERROR",
  "message": "incident_id required",
}
```

Dependency / Reliability

- ไม่เรียก service อื่น
- Idempotent (Safe method)
- Timeout: 30s

Async Contract

Asynchronous Function Contract

Message Contract #1: Request Resource

ข้อมูลทั่วไป

- Message Name: ResourceRequestCreated
- Interaction Style: Request-Async Response (via Message Broker)
- Producer: EmergencyResourceRequest Service
- Consumer: Dispatch Service, Inventory Service
- Channel/Queue: emergency.resource.events.v1
- Version: v1

คำอธิบาย

เมื่อมีผู้ประสบภัยส่งคำร้องขอทรัพยากรเข้ามาใหม่และระบบทำการบันทึกลง Database สำเร็จ บริการนี้จะทำการส่ง Event ออกไปทันที เพื่อแจ้งให้ระบบอื่นรับทราบและนำข้อมูลไปดำเนินการต่อในส่วนของตนเอง (เช่น เช็กสต็อก หรือ จัดทีมกู้ภัย) โดยไม่ต้องรอให้ Service อื่นทำงานเสร็จ เพื่อรักษา High Availability ของระบบรับแจ้งเหตุ

Request

Message Headers

- messageType: ResourceRequestCreated
- eventId: UUID (รหัสอ้างอิงของข้อความนี้ ป้องกันส่งซ้ำ)
- sentAt: ISO-8601 datetime
- traceId: string/uuid (optional - สำหรับใช้ track log ข้าม service)

Message Body

```
{  
  "request_id": "req-1234-uuid",  
  "incident_id": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",  
  "request_for": "CITIZEN",  
  "priority": "CRITICAL",  
  "item": [  
    { "id": "1", "amount": 1 }  
  ],  
  "extra_item": [  
    { "name": "ยาหม่อง", "amount": 3 }  
  ]  
}
```

```
],  
  "location": {  
    "latitude": 16.4425,  
    "longitude": 100.3488  
  },  
  "requested_at": "2026-01-30T14:25:00Z"  
}
```

Field Definition:

Field	Type	Required	Description
request_id	string	Y	รหัสคำร้องขอที่สร้างสำเร็จ (PK จากบริการของเรา)
incident_id	UUID	Y	รหัสอ้างอิงเหตุการณ์ภัยพิบัติ
request_for	string	Y	ประเภทของคำร้องขอว่าสำหรับใคร
priority	string	Y	ความเร่งด่วน: CRITICAL / NORMAL / LOW
items	array<json>	N	รายการสิ่งของที่มีในระบบและจำนวนที่ต้องการ
extra_items	array<json>	N	รายการสิ่งของนอกระบบ (พิมพ์ชื่อเอง) และจำนวนที่ต้องการ
location	json	Y	ตำแหน่งและพิกัดจุดเกิดเหตุ (Latitude, Longitude)
requested_at	datetime	Y	วัน-เวลาที่ส่งคำร้องขอ

Validation Rules

- request_id และ incident_id ต้องไม่เป็นค่าว่าง
- อย่างน้อยต้องมีข้อมูลใน items หรือ extra_items (array ยาวกว่า 0)
- Consumer (ผู้รับ) ต้องใช้ eventId ใน Header เพื่อทำ Idempotency Check ป้องกันการประมวลผลซ้ำซ้อน

Response

Message Headers

- Content-Type: application/json
- Idempotency-Key: <uuid>

Success Message Body

```
{
  "request_id": "req-1234-uuid",
  "incident_id": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",
  "status": "ACCEPTED",
  "processed_by": "DispatchService",
  "message": "Resource request accepted and routed to rescue team"
}
```

Reject/Error Message Body

```
{
  "request_id": "req-1234-uuid",
  "incident_id": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",
  "status": "REJECTED",
  "reasonCode": "OUT_OF_STOCK",
  "message": "Inventory service reports no requested items available in this zone"
}
```

Field Definition:

Field	Type	Required	Description
request_id	string	Y	รหัสคำร้องขอต้นทาง (เพื่อให้เราเอาไปอัปเดต Database ของเราถูกเคส)
incident_id	UUID	Y	รหัสอ้างอิงเหตุการณ์ภัยพิบัติ
status	enum	Y	ผลลัพธ์จากการประมวลผล: ACCEPTED / REJECTED
processed_by	string	N	ชื่อ Service ที่เป็นคนตอบกลับมา (เช่น DispatchService)
reasonCode	string	N	รหัสข้อผิดพลาด (Machine-readable) จะมีค่าเมื่อสถานะเป็น REJECTED
message	string	Y	ข้อความอธิบายผลลัพธ์ (Human-readable) สำหรับเก็บลง Log หรือแสดงผล

Validation Rules

- หาก status เป็น ACCEPTED ควรระบุ processed_by เพื่อให้ทราบว่าใครรับผิดชอบต่อ
- หาก status เป็น REJECTED บังคับว่าต้องมี reasonCode (เช่น OUT_OF_STOCK, OUT_OF_SERVICE_AREA) เพื่อให้ระบบของเราว่าจะต้องจัดการคำขอนี้อย่างไรต่อไป

Service Data

Service Data

1) Resource Request Master Data (Owned by this service)

Field Name	Type	Required	Description	Example
id	UUID (PK)	Y	รหัสอ้างอิงของคำร้องขอ (Primary Key)	req-1234-uuid
incident_id	UUID	Y	รหัสอ้างอิงเหตุการณ์ภัยพิบัติ (เชื่อมกับ Incident Service)	8b9e4b0d-3b7e...
request_for	String	Y	ประเภทของการร้องขอสำหรับใคร	CITIZEN
priority	String (Enum)	Y	ระดับความเร่งด่วน (CRITICAL, NORMAL, LOW)	CRITICAL
requester	JSONB	Y	ข้อมูลผู้แจ้ง (ชื่อ, เบอร์ติดต่อฉุกเฉิน)	{"name": "Somchai", "phone": "012345678"}
location	JSONB	Y	พิกัดจุดเกิดเหตุ และคำอธิบายที่อยู่	{"latitude": 16.4425, "longitude": 100.3488}
status	String (Enum)	Y	สถานะปัจจุบัน (NEW, IN_PROGRESS, CLOSED)	NEW

delivery_team_id	UUID	N	รหัสทีมกู้ภัยที่รับหน้าที่จัดการ (จะอัปเดตภายหลัง)	team-789-uuid
requested_at	Timestamp	Y	วัน-เวลาที่มีการสร้างคำร้องขอนี้ขึ้นมา	2026-01-30T14:25:00Z

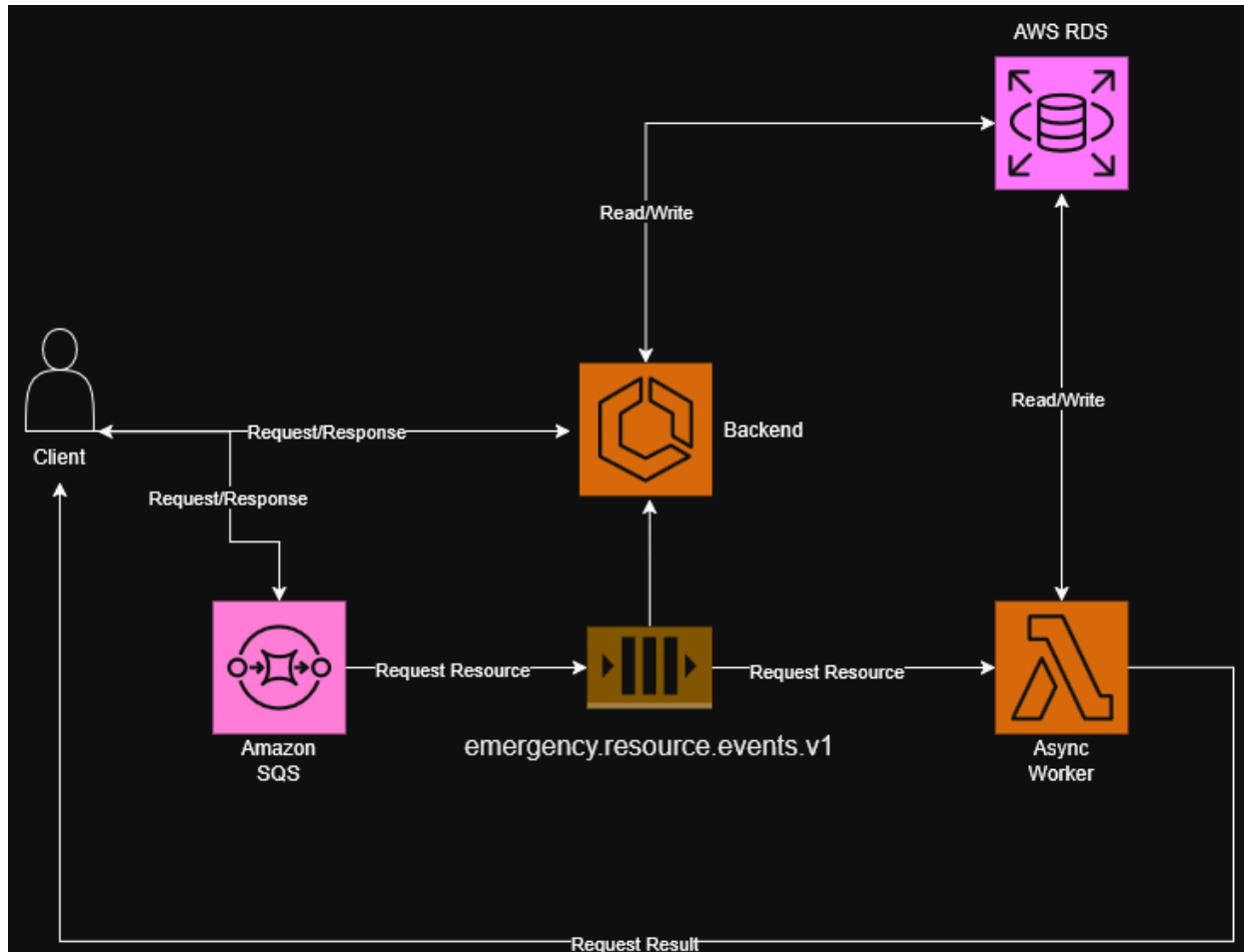
*items และ extra_items อนุญาตให้เป็น Null ได้ แต่ในการ Validation ของ API บังคับว่าต้องมีข้อมูลอย่างน้อยใน 1 ฟิลด์)

2) Request Line Item (Owned by this service)

Field Name	Type	Required	Description	Example
id	UUID (PK)	Y	รหัสอ้างอิงของรายการย่อย (Primary Key)	item-8899-uuid
request_id	UUID (FK)	Y	รหัสคำร้องขอหลัก	req-1234-uuid
item_type	String	Y	ประเภทสิ่งของ (STANDARD = มีในระบบ, EXTRA = นอกกระบบ)	EXTRA
item_ref_id	String	N	รหัสสิ่งของอ้างอิง (กรณีเป็น STANDARD)	1 (ปล่อยว่างถ้าเป็น EXTRA)
item_name	String	N	ชื่อสิ่งของ (กรณีเป็น EXTRA ที่ผู้ใช้พิมพ์เอง)	"ยาหม่อง"

amount_requested	Integer	Y	จำนวนที่ร้องขอ	3
------------------	---------	---	----------------	---

Service Architecture



Service Architecture

Components

- **Client**: ผู้เรียกใช้งานระบบ (เช่น ประชาชนผู้ประสบภัย หรือเจ้าหน้าที่ส่วนหน้า) ที่ส่งคำขอและรอรับผลลัพธ์
- **Backend**: บริการหลัก (Core Service) ทำหน้าที่รับคำขอแบบ Synchronous จาก Client รวมถึงสามารถดึงข้อความ/เหตุการณ์จากคิวมาประมวลผลได้
- **AWS RDS**: ฐานข้อมูลเชิงสัมพันธ์ (Relational Database) สำหรับจัดเก็บข้อมูลที่ EmergencyResourceRequest Service เป็นเจ้าของ (เช่น ข้อมูลคำร้องขอ, สถานะคำขอ และประวัติการอัปเดต)
- **Amazon SQS**: จุดรับคำขอแบบ Asynchronous (เช่น คำร้องขอทรัพยากร) ที่ถูกส่งเข้ามาจาก Client เพื่อลดภาระการรอคอยของระบบ
- **Message Queue (emergency.resource.events.v1)**: คิวสำหรับพักและจัดการ Event "Request Resource" ก่อนที่จะกระจายงานไปให้ส่วนอื่นๆ ประมวลผล
- **Async Worker (AWS Lambda)**: ฟังก์ชันที่ทำงานอยู่เบื้องหลัง ทำหน้าที่ดึงข้อความจากคิวมาประมวลผล อ่าน/เขียนข้อมูลลงในฐานข้อมูล และแจ้งผลลัพธ์กลับไปยัง Client โดยตรง

Explanation สถาปัตยกรรมของ EmergencyResourceRequest Service ถูกออกแบบให้รองรับทั้งการทำงานแบบประสานเวลา (Synchronous) และไม่ประสานเวลา (Asynchronous) โดยใช้ฐานข้อมูลส่วนกลางคือ **AWS RDS** ในการจัดเก็บข้อมูลสถานะคำร้องขอ

การทำงานแบ่งออกเป็น 2 เส้นทางหลัก ดังนี้:

1. **Synchronous Flow (สำหรับงานที่ต้องการผลลัพธ์ทันที):** Client สามารถสื่อสารโดยตรงกับ **Backend** (ผ่านรูปแบบ Request/Response) เพื่อดูข้อมูลหรือตรวจสอบสถานะคำขอ โดย Backend จะทำการอ่านและเขียนข้อมูล (Read/Write) กับ AWS RDS แล้วตอบกลับไปยัง Client ทันที รูปแบบนี้เหมาะสำหรับการดึงข้อมูล (Query) เพื่อแสดงผลบนหน้าจอ
2. **Asynchronous Flow (สำหรับงานสร้างคำร้องขอ หรือประมวลผลที่ใช้เวลา):** เมื่อ Client ต้องการส่งคำร้องขอทรัพยากร (Request Resource) คำขอนั้นจะถูกส่งไปเข้าคิวที่ **Amazon SQS** และจัดเก็บในคิว **emergency.resource.events.v1** ซึ่งช่วยให้ Client ไม่ต้องรอระบบประมวลผลจนเสร็จ จากนั้น **Async Worker (AWS Lambda)** จะทำการดึงข้อมูลจากคิวไปประมวลผล ทำการบันทึกหรืออัปเดตข้อมูลลงใน **AWS RDS** เมื่อทุกอย่างเสร็จสมบูรณ์ Async Worker จะส่งผลลัพธ์การดำเนินการ (Request Result) กลับไปแจ้งให้ Client ทราบ (เช่น ผ่าน Push Notification หรือ Webhook) นอกจากนี้ ตัว Backend เองก็สามารถรับ Event จากคิวนี้ไปใช้ประโยชน์ในการอัปเดตสถานะของระบบย่อยอื่นๆ ได้เช่นเดียวกัน รูปแบบนี้ช่วยลดคอขวดของระบบและทำให้รองรับคำขอจำนวนมากพร้อมกันในสถานการณ์ภัยพิบัติได้อย่างมีประสิทธิภาพ

Service Interaction

Dependency Mapping

